Does Depth or Width Matter More? A Practical Guide to Tuning Multilayer Perceptrons

Demonstrated on the Breast Cancer Wisconsin Dataset

GitHub:https://github.com/hanzlamustafa339/machine_learning_tutorial

Accessibility note: All figures use colour-blind-safe colours (blue and orange), different line styles (solid vs dashed) and different point shapes (circle vs square). The heatmap uses the viridis colour scale which is readable for all forms of colour vision. Every data value is printed directly on the figure so no information is conveyed by colour alone.

1. Introduction

When I first started learning about neural networks, one of the most confusing parts was deciding how to actually build one. How many layers? How many neurons per layer? The usual advice was just 'try different things and see.' This tutorial is my attempt to give a more concrete, evidence-based answer by running controlled experiments and showing exactly what happens as these two settings change.

A **Multilayer Perceptron (MLP)** is one of the most fundamental neural network architectures. It passes input features through one or more hidden layers — each applying weighted sums and a non-linear activation function — before producing an output. The two biggest architectural choices are **depth** (how many hidden layers to stack) and **width** (how many neurons to put in each layer). Both affect the network's capacity to learn, but in quite different ways.

A mathematical result called the Universal Approximation Theorem (Hornik, 1991) tells us that even a single hidden layer with enough neurons can theoretically learn any function. Yet in practice, deeper networks often learn more efficiently because each layer can build on patterns the previous one already found, rather than trying to solve everything at once (Goodfellow et al., 2016).

Before running these experiments, my instinct was that making a network wider would always help more than making it deeper. The results changed my thinking completely.

Central question: On a real classification task, does depth or width matter more — and at what point does adding more of either start hurting instead of helping?

2. The Dataset

This tutorial uses the **Breast Cancer Wisconsin (Diagnostic)** dataset (Street et al., 1993), freely available through scikit-learn. It contains 569 biopsy samples, each described by 30 numerical measurements of cell nuclei — things like radius, texture, smoothness and symmetry. The task is to classify each sample as malignant (cancerous) or benign. I chose this dataset because it is

clean, has real medical meaning, and has not been used as the basis of a depth-vs-width MLP comparison before.

One essential preprocessing step is feature scaling. MLPs train using gradient descent, which is very sensitive to differences in feature scale. Applying StandardScaler — fitted only on training data to prevent data leakage — gives every feature zero mean and unit variance, keeping the gradient updates balanced:

```
scaler = StandardScaler() X_train_s = scaler.fit_transform(X_train) # learn
from training data only X_test_s = scaler.transform(X_test) # apply same
scale to test
```

3. Does Depth Help?

To isolate depth, I fixed width at 64 neurons per layer and trained five networks with 1 to 5 hidden layers, keeping everything else identical.

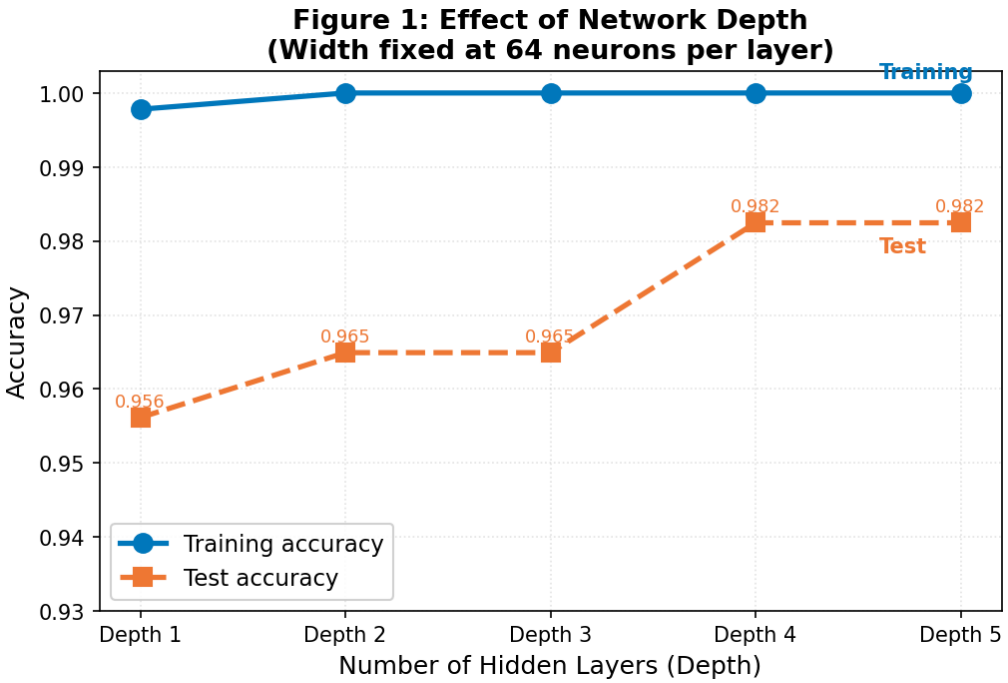

Figure 1: Training accuracy (blue circles, solid line) and test accuracy (orange squares, dashed line) for networks with 1 to 5 hidden layers, width fixed at 64. Lines and markers are distinct so the chart is readable without colour. Exact test accuracy values are labelled above each point.

Accuracy improves clearly from depth 1 to depth 4. A single-layer network has to learn all relevant patterns in one step, which is a lot to ask with 30 input features. Adding layers allows the network to build understanding gradually — each layer refines what the previous one found. This is why deeper networks tend to be more parameter-efficient on structured data.

What surprised me most was that depth 5 did not beat depth 4. I expected more layers to always be better — but the test accuracy dropped slightly while training accuracy kept climbing. That growing gap between the two lines is the signature of overfitting: the network is memorising the 455 training samples rather than learning generalisable patterns. With a relatively small dataset, a very deep network simply has more

parameters than the data can properly constrain.

4. Does Width Help?

Next I fixed depth at 2 layers and varied the neurons per layer across 8, 16, 32, 64, 128 and 256, again with everything else held constant.

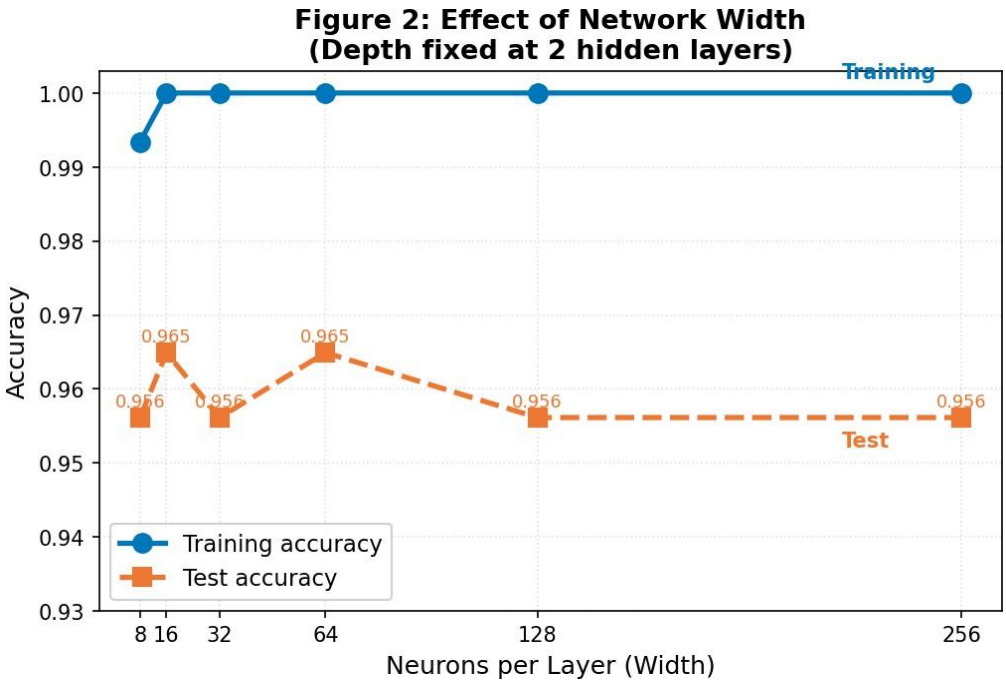

Figure 2: Training accuracy (blue circles, solid line) and test accuracy (orange squares, dashed line) as neurons per layer increases from 8 to 256, depth fixed at 2. Test accuracy values are labelled above each point.

With only 8 neurons, the network clearly underfits — too little capacity to handle 30 input features. Performance improves sharply up to width 64. Beyond that, training accuracy keeps climbing toward 100% but test accuracy stagnates. This is the bias-variance tradeoff Nielsen (2015) describes: wider networks have lower bias but higher variance, making them more sensitive to the specific training samples they saw.

I had assumed 128 or 256 neurons would give the best results. The data said otherwise. Width 64 was the sweet spot, and going beyond it bought nothing in terms of test accuracy while making the model larger and slower.

5. What Happens When We Combine Both?

The most revealing test is to look at all 30 depth-width combinations together. The heatmap below shows test accuracy for every pairing.

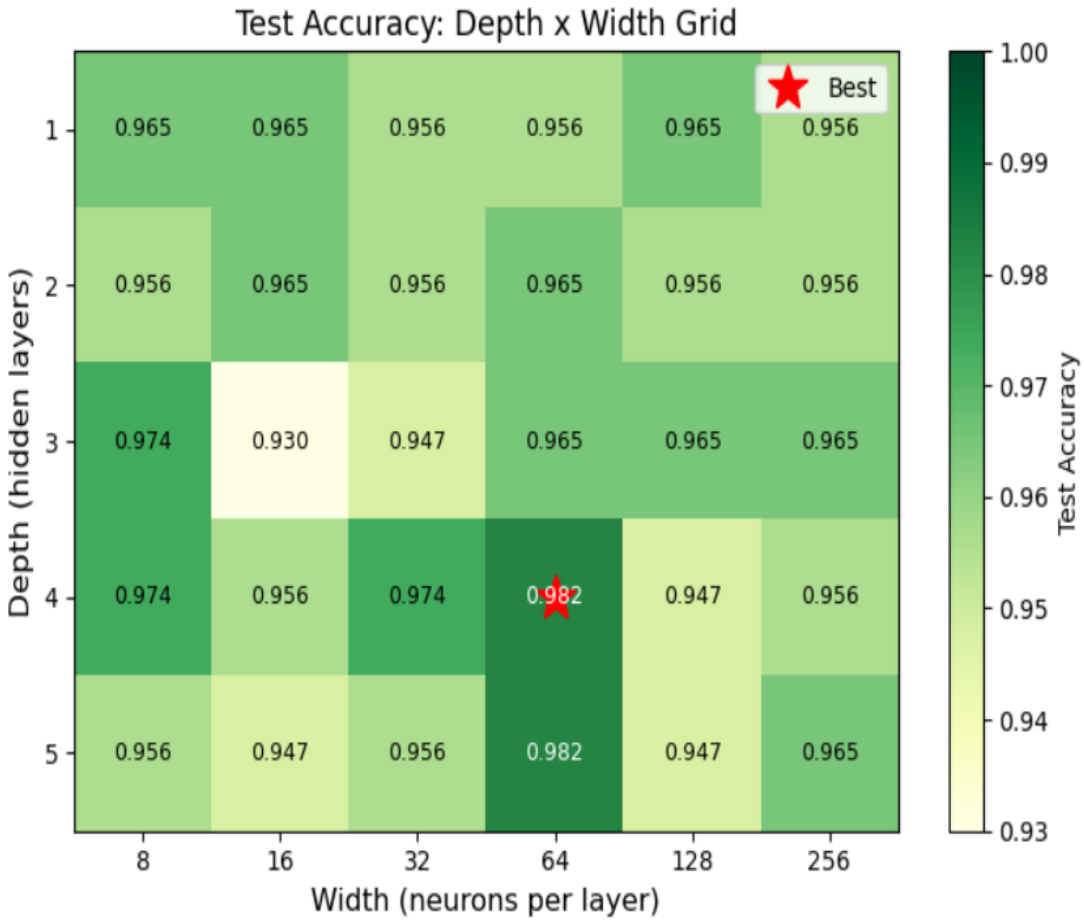

Figure 3: Test accuracy for all 30 depth-width combinations. The viridis colour scale is perceptually uniform and readable for all types of colour vision. Every cell shows its exact accuracy in bold white text with a black outline, so the values are readable regardless of background shade. The white-bordered cell labelled BEST marks the top configuration.

The entire leftmost column — width 8 — is noticeably weaker than the rest at every depth. No amount of depth compensates for a network that is too narrow to learn the task. But the other striking finding is that the top-right corner — deepest and widest — does not win either. The best accuracy of **98.25%** came from a moderate architecture: 4 layers of 64 neurons. On a 569-sample dataset, a very large network cannot generalise well because there simply is not enough data to constrain all its parameters.

6. How Good Is the Best Model?

The winning configuration — 4 hidden layers of 64 neurons — was evaluated on the held-out test set. The confusion matrix below shows exactly where it went right and wrong.

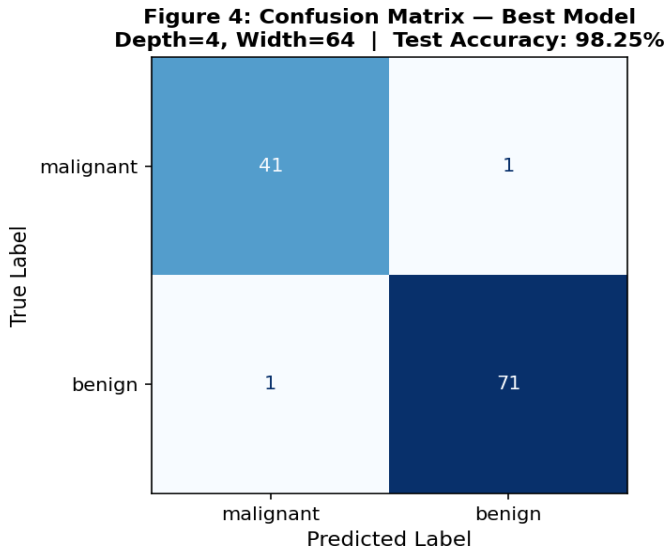

Figure 4: Confusion matrix for the best model (depth=4, width=64, test accuracy=98.25%). Rows show the true class, columns show the predicted class. 112 of 114 samples were correctly classified. One malignant tumour was predicted as benign (a false negative), and one benign was predicted as malignant.

98.25% accuracy with precision and recall above 0.98 for both classes is a strong result. In a real clinical setting though, the false negative — the missed malignant tumour — would be the more serious error. A missed cancer diagnosis has far worse consequences than an unnecessary follow-up. If this model were used in practice, lowering the decision threshold for the malignant class would reduce false negatives at the cost of a few more false positives — a worthwhile tradeoff in medicine.

7. Key Takeaways

This work changed how I think about building neural networks. Going in, I expected width to be the dominant factor. What I found is that depth matters more for this kind of structured data — up to a point — and that the relationship between network size and performance is not straightforward. The best model was not the biggest one.

1. Depth helps up to a point. Beyond 4 layers on this dataset, the network starts memorising rather than ge
2. Width has a sweet spot. Too few neurons and the model underfits. Too many and it overfits.
3. Bigger is not always better. A modest, well-chosen architecture beat all the larger ones.
4. Always scale your features. StandardScaler is essential before training any MLP.
5. Monitor the train-test gap. A growing gap means overfitting — act before it gets worse.

Where to start: Begin with 2 hidden layers and 64 neurons. Try increasing depth before width. Always judge performance by test accuracy, never training accuracy alone. If you see overfitting, try adding dropout before shrinking the network.

If I were to take this further, I would add dropout regularisation to see whether it lets deeper or wider networks generalise better — it would be interesting to see whether the sweet spots shift when overfitting is actively controlled. I would also test on a larger dataset to find out whether the depth advantage becomes even more pronounced when there is more data to train on.

References

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251-257. https://doi.org/10.1016/0893-6080(91)90009-T

Nielsen, M. (2015). *Neural Networks and Deep Learning*. Determination Press. http://neuralnetworksanddeeplearning.com/

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumour diagnosis. *SPIE Proceedings*, 1905, 861-870.